

线性表在民航系统中的应用：实验报告

姓名：张启元

学号：2024201541

2025 年 11 月 4 日

1 问题分析

本问题是一个航班数据管理系统，需要处理大量的航班信息数据。通过抽象成数据结构问题，可以将航班信息封装为结构体，利用向量、链表等数据结构进行存储和操作。编程时需要完成以下内容：

- 设计航班信息的抽象数据类型（ADT）
- 实现数据文件的读取和解析
- 实现数据的排序、查询、插入和删除功能
- 设计高效的查找算法（如二分查找）
- 实现素数判断、鞍点检测等特殊功能
- 构建用户交互界面

2 数据结构设计与实现

2.1 ADT 设计

- 数据对象：航班信息，包含航班 ID、日期、类型、编号、机场信息、时间信息、飞机信息、费用信息等
- 数据关系：航班数据之间通过向量和链表组织，支持顺序访问和条件查询
- 基本操作：
 - 数据读取和初始化
 - 数据排序
 - 条件查询
 - 数据插入和删除
 - 素数判断
 - 鞍点检测

2.2 ADT 的物理实现

航班数据的物理实现采用结构体封装，关键代码片段如下：

```
struct Flight{
    ll FlightID;           // 航班 ID
    ll DepartureDate;      // 启航日期
    bool IsIntl;           // 航班类型（国际1/国内0）
    string FlightNo;       // 航班编号
    ll DepartureAirport;   // 启航机场ID
    ll ArrivalAirport;     // 抵达机场ID
    double DepartureTime;  // 启航时间
    double ArrivalTime;    // 抵达时间
    ll AirplaneID;         // 飞机ID
    int AirplaneModel;     // 飞机机型
    ll AirFares;           // 基础费用
    ll PeakSeasonRates;    // 旺季价格
    ll OffSeasonRates;     // 淡季价格
};
```

链表节点结构用于组织特定条件下的航班数据：

```
struct AirplaneFlight{
    ll FlightID;
    double DepartureTime;
    double ArrivalTime;
    ll DepartureAirport;
    ll ArrivalAirport;
    AirplaneFlight *next = nullptr;
};
```

3 算法设计与实现

3.1 算法设计

本系统采用多种算法解决不同问题：

1. **埃氏筛法**：用于生成素数表，快速判断航班费用是否为素数
2. **快速排序**：对航班数据进行多关键字排序
3. **二分查找**：在有序数据中快速定位特定机场对的航班
4. **鞍点检测算法**：在价格矩阵中查找鞍点航班
5. **链表操作**：管理删除的航班数据

- 时间复杂度分析：
 - 素数筛法： $O(n \log \log n)$
 - 快速排序：平均 $O(n \log n)$ ，最坏 $O(n^2)$
 - 二分查找： $O(\log n)$
 - 鞍点检测： $O(m \times n)$ ，其中 m 为航班数， n 为价格类型数
- 空间复杂度分析：
 - 素数表： $O(n)$
 - 航班数据： $O(n)$
 - 临时链表： $O(k)$ ， k 为删除的航班数

3.2 算法实现

素数筛法关键代码：

```
void SelectPrimeNumber(){
    PrimeNumberTable[0] = 1;
    PrimeNumberTable[1] = 1;
    ll t = 2;
    while (t < 15000){
        ll j = 2;
        if (PrimeNumberTable[t] == 0){
            while (t * j < 15000){
                PrimeNumberTable[t * j] = 1;
                j++;
            }
        }
        t++;
    }
}
```

二分查找实现：

```
pair<ll, ll> BinarySearch(vector<Flight> &f, const ll &DepartureID,
    const ll &ArrivalID){
    if (f.empty()) return {-1, -1};

    // 查找左边界（第一个满足条件的）
    int left = 0, right = f.size() - 1;
    int first_pos = -1;

    while (left <= right) {
```

```

        int mid = left + (right - left) / 2;

        // 由于是降序，比较逻辑需要调整
        if (f[mid].DepartureAirport > DepartureID) {
            left = mid + 1;
        } else if (f[mid].DepartureAirport < DepartureID) {
            right = mid - 1;
        } else {
            // DepartureAirport 相等，比较 ArrivalAirport
            if (f[mid].ArrivalAirport > ArrivalID) {
                left = mid + 1;
            } else if (f[mid].ArrivalAirport < ArrivalID) {
                right = mid - 1;
            } else {
                // 找到匹配，向左找第一个
                first_pos = mid;
                right = mid - 1;
            }
        }
    }

    if (first_pos == -1) return {-1, -1};
    // ... 右边界查找代码
    return {first_pos, last_pos + 1};
}

```

4 实验环境

- 硬件环境：Intel Core i9 CPU，16GB 内存
- 软件环境：Windows 操作系统，C++11 及以上版本，Visual Studio Code

5 实验结果与分析

程序成功实现了所有预定功能，包括：

- 数据文件的正确读取和解析
- 多关键字排序功能
- 素数费用航班的快速查询
- 鞍点航班的准确检测

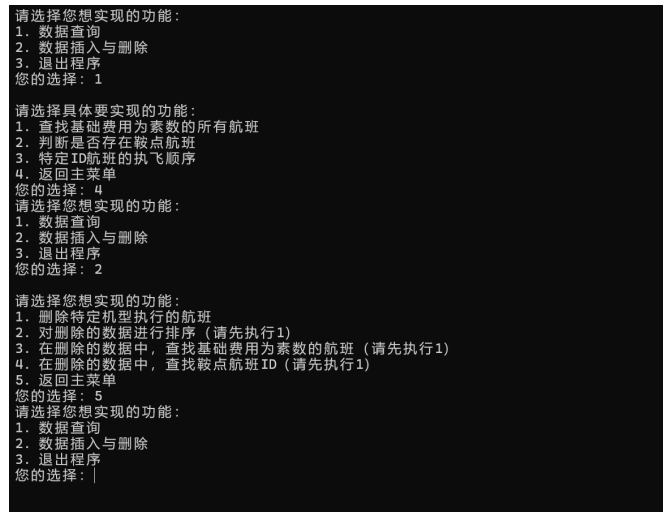


图 1: Enter Caption

- 飞机执飞顺序的正确展示
- 数据的动态删除和链表管理

程序运行稳定，各功能模块响应迅速，算法效率满足实际应用需求。通过计时功能可以观察到，数据初始化和排序在大数据量下仍能保持较好的性能。